PKCERT AI & Software Development Internship Task 04 — NumPy & Pandas for Data Analysis

Abdullah Amir

AI and Software Development
07 July 2026

Contents

Part A — NumPy Fundamentals

1. Creating One- and Multi-Dimensional Arrays

NumPy's whole world revolves around one object: the `ndarray`. Unlike a plain Python list, every element shares the same type and the data sits in one contiguous block of memory, which is what makes the maths on it so fast. There are several ways to build one, and I usually pick whichever matches how much I already know about the contents.

```
import numpy as np

a = np.array([1, 2, 3, 4])             # 1-D array from a list
b = np.array([[1, 2, 3], [4, 5, 6]])   # 2-D array (2 rows, 3 cols)

zeros = np.zeros((2, 3))      # all zeros, shape 2x3
ones = np.ones((3, 3))        # all ones, shape 3x3
rng = np.arange(0, 10, 2)     # [0 2 4 6 8]
lin = np.linspace(0, 1, 5)    # 5 evenly spaced values 0..1
eye = np.eye(3)               # 3x3 identity matrix

print(b.shape, b.ndim, b.dtype)    # (2, 3) 2 int64
```

Listing 1: Different ways to create NumPy arrays.

The three attributes I check most often are `shape` (the size along each dimension), `ndim` (how many dimensions there are), and `dtype` (the element type). Knowing these up front saves a lot of confusing shape errors later.

2. Indexing, Slicing, Reshaping, and Mathematical Operations

Indexing and slicing feel familiar coming from lists, but with more than one dimension you separate the axes with a comma. Reshaping rearranges the same data into a new shape without copying it, and the arithmetic operators all work element by element.

```
arr = np.arange(1, 13)          # [1 2 ... 12]
```

```

# --- reshape into 3 rows x 4 cols ---
grid = arr.reshape(3, 4)

# --- indexing and slicing (row, column) ---
print(grid[0, 0])      # 1   -> first element
print(grid[1, :])      # [5 6 7 8]  -> whole second row
print(grid[:, 2])      # [3 7 11]   -> third column
print(grid[0:2, 1:3]) # top-left block

# --- element-wise maths ---
print(grid + 10)       # add 10 to every element
print(grid * 2)        # double every element
print(grid.sum(axis=0))   # column sums
print(grid.mean(axis=1))  # row averages
```

Listing 2: Indexing, slicing, reshaping, and element-wise maths.

The `axis` argument is worth pausing on: `axis=0` runs down the rows (so you get one result per column) and `axis=1` runs across the columns (one result per row). I mix these up less once I picture the axis being the one that "collapses".

3. Broadcasting

Broadcasting is NumPy's rule for doing maths on arrays whose shapes don't match exactly. Instead of forcing you to manually copy a smaller array out to the bigger one's size, NumPy stretches it for you behind the scenes, without actually using the extra memory.

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])      # shape (2, 3)
row = np.array([10, 20, 30])        # shape (3,)

# 'row' is broadcast across both rows of 'matrix'
print(matrix + row)
# [[11 22 33]
#  [14 25 36]]

# a scalar is the simplest broadcast of all
print(matrix * 2)

# scaling each column by a column vector
col = np.array([[1], [2]])          # shape (2, 1)
print(matrix * col)
```

Listing 3: Broadcasting a 1-D array across a 2-D array.

The advantage is both cleaner code and better performance. There's no explicit loop to write, nothing gets duplicated in memory, and the operation still runs at C speed. The rule itself is simple: NumPy lines the shapes up from the right, and two dimensions are compatible when they're equal or one of them is 1.

4. Vectorized Operations vs. Traditional Loops

This is the reason people reach for NumPy in the first place. A vectorized operation applies to the whole array at once in optimised C code, whereas a Python `for` loop pays the interpreter's overhead on every single element.

```python
import numpy as np
import time

n = 1_000_000
data = np.arange(n)

# --- traditional loop ---
start = time.time()
loop_result = [x * 2 for x in data]
print("Loop:", time.time() - start, "seconds")

# --- vectorized ---
start = time.time()
vec_result = data * 2
print("Vectorized:", time.time() - start, "seconds")
```

Listing 4: Timing a vectorized operation against a plain loop.

On my machine the vectorized version finished many times faster than the loop for the same million elements. Beyond the speed, the code reads better too: `data * 2` states the intent directly, with no loop boilerplate to wade through.

5. Linear Algebra Operations

NumPy also covers the linear-algebra basics that turn up all the time in machine learning. Dot products, matrix multiplication, transposes, and inverses are all one call away.

```python
A = np.array([[1, 2],
              [3, 4]])
B = np.array([[5, 6],
              [7, 8]])

v1 = np.array([1, 2, 3])
v2 = np.array([4, 5, 6])

print(np.dot(v1, v2))   # 32  -> dot product of two vectors
print(A @ B)            # matrix multiplication (the @ operator)
print(A.T)              # transpose (rows become columns)

inv = np.linalg.inv(A)  # inverse (only for square, non-singular A)
print(inv)
print(A @ inv)          # ~ identity matrix, confirms the inverse
```

Listing 5: Dot product, matrix multiply, transpose, and inverse.

A couple of things I keep in mind here: `*` is element-wise multiplication, while `@` (or `np.dot`) is true matrix multiplication — an easy trap to fall into. And the inverse only exists for a square matrix whose determinant isn't zero, otherwise `np.linalg.inv` raises an error.

Part B — Pandas Fundamentals

1. Series and DataFrames

Pandas builds on top of NumPy and gives you two main objects. A `Series` is a one-dimensional labelled array — think of a single column with an index. A `DataFrame` is the two-dimensional table you get by putting several of those columns side by side, which is how most real data actually looks.

```python
import pandas as pd

# a Series: values with a custom index
marks = pd.Series([85, 72, 90], index=["Ali", "Sara", "Omar"])
print(marks["Sara"])   # 72

# a DataFrame built from a dictionary of columns
df = pd.DataFrame({
    "name":   ["Ali", "Sara", "Omar", "Hina"],
    "age":    [22, 25, 23, 24],
    "dept":   ["CS", "EE", "CS", "EE"],
    "salary": [50000, 62000, 48000, 55000],
})

print(df.head())      # first few rows
print(df.info())      # column names, types, non-null counts
print(df.describe()) # quick summary stats for numeric columns
```

Listing 6: Creating a Series and a DataFrame.

2. Indexing, Filtering, Sorting, and Selection

Once the data is in a DataFrame, most of the work is pulling out the parts you care about. I use `loc` for label-based selection and `iloc` for position-based selection, and boolean masks for filtering by a condition.

```python
# --- selection ---
print(df["name"])             # a single column (a Series)
print(df[["name", "age"]])    # several columns
print(df.loc[0])              # row by label
print(df.iloc[0:2])           # first two rows by position

# --- filtering with a boolean mask ---
cs_staff = df[df["dept"] == "CS"]
well_paid = df[df["salary"] > 50000]
both = df[(df["dept"] == "EE") & (df["age"] < 25)]

# --- sorting ---
print(df.sort_values("salary", ascending=False))
```

Listing 7: Selecting, filtering, and sorting rows.

3. GroupBy Operations

The `groupby` pattern is the one I use most for summarising. It follows a "split–apply–combine" idea: split the rows into groups, apply an aggregation to each group, then combine the results back into a table.

```python
# average salary per department
print(df.groupby("dept")["salary"].mean())

# several aggregations at once
summary = df.groupby("dept").agg(
    avg_salary=("salary", "mean"),
    max_age=("age", "max"),
    headcount=("name", "count"),
)
```

```
print(summary)
```

Listing 8: Summarising with groupby.

4. Merging and Joining DataFrames

Real data is usually spread across more than one table, so being able to stitch them together on a shared key is essential. `pd.merge` works much like a SQL join, and the `how` argument decides which rows survive.

```
employees = pd.DataFrame({
    "emp_id": [1, 2, 3],
    "name":   ["Ali", "Sara", "Omar"],
})
salaries = pd.DataFrame({
    "emp_id": [1, 2, 4],
    "salary": [50000, 62000, 47000],
})

inner = pd.merge(employees, salaries, on="emp_id", how="inner")
left  = pd.merge(employees, salaries, on="emp_id", how="left")
outer = pd.merge(employees, salaries, on="emp_id", how="outer")
```

Listing 9: Merging two DataFrames on a shared key.

The `how` argument is the key to reading the result. An **inner** join keeps only the `emp_id` values found in both tables (so 1 and 2). A **left** join keeps every employee and fills a missing salary with `NaN` (Omar has no salary row). An **outer** join keeps everything from both sides, filling the gaps on either side with `NaN`. Choosing the right one comes down to whether you're willing to drop unmatched rows.

Part C — Data Analysis Mini Project

Overview

For the mini project I used the classic **Titanic dataset**, a public CSV listing the passengers on the RMS Titanic along with whether each one survived. It's a good choice for practising exploratory analysis because it mixes numeric columns (age, fare) with categorical ones (sex, passenger class) and, importantly, has real missing values to clean up. The application reads the file, cleans it, produces summary statistics, and then digs into what actually drove survival.

Reading the Data and a First Look

```
import pandas as pd

df = pd.read_csv("titanic.csv")

print(df.shape)              # (891, 12) -> rows, columns
print(df.head())             # first five passengers
print(df.info())             # dtypes and non-null counts
print(df.isnull().sum())     # how many values are missing per column
```

Listing 10: Loading the dataset and inspecting it.

The dataset has 891 rows. The null count immediately shows where the problems are: `Age` is missing for a good number of passengers, `Cabin` is missing for most of them, and `Embarked` has just a couple of gaps.

Cleaning the Missing Values

My cleaning strategy was decided column by column, based on how much was missing and how useful the column was.

```
# Age: fill gaps with the median (robust to outliers)
df["Age"] = df["Age"].fillna(df["Age"].median())

# Embarked: only a couple missing, so use the most common port
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

# Cabin: too many missing to be useful, so drop the whole column
df = df.drop(columns=["Cabin"])

# confirm there are no missing values left
print(df.isnull().sum())
```

Listing 11: Handling the missing values.

The reasoning: I filled `Age` with the *median* rather than the mean because a few very old or very young passengers would skew the mean, whereas the median holds steady. For `Embarked` only two values were missing, so filling them with the most frequent port (the mode) is safe and keeps the rows. `Cabin` was missing for the large majority of passengers, so imputing it would be mostly guesswork — dropping the column was the honest choice.

Summary Statistics and Exploratory Analysis

With clean data I moved on to the questions the dataset is famous for, using `groupby` to slice survival by different factors.

```
# overall survival rate
print(df["Survived"].mean())          # ~0.38

# survival by passenger class
print(df.groupby("Pclass")["Survived"].mean())

# survival by port of embarkation
print(df.groupby("Embarked")["Survived"].mean())

# average fare paid per class
print(df.groupby("Pclass")["Fare"].mean())

# bucket age into groups, then survival by age group
df["AgeGroup"] = pd.cut(df["Age"],
    bins=[0, 12, 18, 35, 60, 100],
    labels=["Child", "Teen", "Young Adult", "Adult", "Senior"])
print(df.groupby("AgeGroup", observed=True)["Survived"].mean())
```

Listing 12: Exploratory analysis with groupby.

Key Findings

- Only about **38%** of passengers survived overall.
- **Passenger class was a major factor.** First-class passengers survived at roughly 63%, second class at about 47%, and third class at only 24% — a steep drop from top to bottom.
- **Fare tracks class closely.** First-class passengers paid far more on average (around £84) than third-class (around £14), which lines up with them being berthed closer to the lifeboats.

- **Age mattered too.** Children had the best odds by age group (about 58%), while seniors had the worst (about 23%).
- Combining class and age showed the effects stacking up: a first- or second-class child was very likely to survive, whereas an older third-class passenger was among the least likely.

The full project — the cleaning script, the analysis code, and a `README.md` describing the dataset, the cleaning process, the analysis, and these findings — is uploaded to my GitHub repository for the internship.

Conclusion

This task tied together the numerical and data-handling side of Python. NumPy gave me the foundation — arrays, broadcasting, and vectorization that runs far faster than hand-written loops — while Pandas built on top of it with Series, DataFrames, and the filtering, grouping, and merging tools that make real datasets manageable. Pulling it all together in the Titanic mini project showed how the pieces fit: load messy data, clean it thoughtfully, and then let `groupby` surface the story hidden inside it. These are the exact skills the rest of the AI/ML work in the internship will keep leaning on.